GitHub Actions et GitLab CI : syntaxe, fonctionnement et comparaison

La CI/CD (Continuous Integration / Continuous Delivery : intégration et livraison continues) consiste à exécuter automatiquement, à chaque modification du code, la construction, les tests et le déploiement. GitHub Actions et GitLab CI font la même chose avec des vocabulaires et des modèles d'exécution différents : le YAML (YAML Ain't Markup Language) d'un côté ne se transpose pas mot à mot de l'autre.

> **Choix du projet : GitHub et GitHub Actions** pour le dépôt, la CI et le registre d'images. Le dépôt contient un workflow (ci.yml) mais **aucun** `.gitlab-ci.yml` : GitLab CI n'est ici qu'un **sujet de comparaison**. Ses parties sont de la théorie non vérifiée dans ce dépôt, et l'exemple de transposition est une illustration, pas un fichier du projet.

Vocabulaire côte à côte

Notion	GitHub Actions	GitLab CI
Fichier	`.github/workflows/*.yml` (plusieurs possibles)	`.gitlab-ci.yml` à la racine (un seul, découpable avec `include:`)
Ensemble d'une exécution	workflow (« run »)	pipeline
Unité de travail	job	job
Ordre	jobs en parallèle par défaut, `needs:` pour les enchaîner	`stages:` séquentiels ; les jobs d'un même stage tournent en parallèle ; `needs:` pour sauter les stages
Étape dans un job	`steps:` (une action `uses:` ou une commande `run:`)	`script:` (liste de commandes)
Brique réutilisable	action (`uses: actions/checkout@v7`)	`include:`, `extends:`, composants CI/CD
Machine d'exécution	`runs-on:` (runner hébergé ou auto-hébergé)	runner choisi par `tags:`, avec un executor (Docker, shell…)
Image de conteneur	`container:` ou `services:`	`image:` et `services:`
Déclencheurs	`on:` (`push`, `pull_request`, `schedule`…)	`rules:` / `workflow:rules:`
Condition	`if:` avec expressions `${{ }}`	`rules: - if:` avec variables `$CI_...`
Secrets et variables	`secrets.X`, `vars.X`, `env:`	variables CI/CD (masquées ou protégées) dans les paramètres, `variables:`

Notion	GitHub Actions	GitLab CI
Jeton d'accès	`GITHUB_TOKEN` (droits réglés par `permissions:`)	`CI_JOB_TOKEN`
Fichiers entre jobs	`actions/upload-artifact` / `download-artifact`	`artifacts:` (transmis automatiquement aux stages suivants)
Cache	`cache:` des actions `setup-*` ou `actions/cache`	`cache:` avec une `key:`
Matrice	`strategy.matrix`	`parallel:matrix`
Annuler les doublons	`concurrency:` + `cancel-in-progress`	`interruptible: true` (+ auto-cancel des pipelines redondants)
Registre d'images	GitHub Container Registry (`ghcr.io`)	GitLab Container Registry (`$CI_REGISTRY_IMAGE`)

Différence de fond : où vivent les fichiers

- **GitHub Actions** : tous les `steps` d'un job s'exécutent sur **la même machine, dans le même dossier de travail**. Un fichier créé par une étape est lu par la suivante. Pour passer des fichiers à un autre job, il faut un artefact.
- **GitLab CI** : chaque **job démarre dans un environnement vierge** (dépôt récupéré de nouveau). Ce qu'un job doit transmettre aux suivants passe par `artifacts:`, et ce qu'on veut réutiliser d'une exécution à l'autre (dépendances) par `cache:`.

GitHub Actions en bref

```yaml
on: [push, pull_request]        # déclencheurs
permissions: { contents: read } # droits du GITHUB_TOKEN
jobs:
  test:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v7     # action réutilisable
      - run: ./run-tests.sh           # commande shell
      - if: always()                  # même si l'étape précédente a échoué
        uses: actions/upload-artifact@v7
```

GitLab CI en bref

```yaml
stages: [test, build]
test:
  stage: test
  image: node:24-alpine           # chaque job choisit son image
  cache: { key: { files: [package-lock.json] }, paths: [.npm/] }
  script:
    - npm ci --cache .npm
    - ./run-tests.sh
  artifacts:
    when: always                  # équivalent de "if: always()" pour les fichiers
    paths: [test-results/]
    reports: { junit: test-results/xml/*.xml }
```

Dans ce dépôt

Le workflow ci.yml est un exemple de GitHub Actions soigné :

- Déclencheurs, lignes 8-11 : `push` et `pull_request`.
- Droits minimaux, ligne 14 : `permissions` limite le `GITHUB_TOKEN` à la lecture du code, plus l'écriture des « checks » pour publier le rapport.
- Annulation des doublons, ligne 19 : `concurrency` avec `cancel-in-progress: true`.
- Version du runner fixée, ligne 25 : `ubuntu-24.04` plutôt que `latest`, pour un environnement stable.
- Actions fixées sur un SHA, ligne 27 : on référence le commit exact (SHA, Secure Hash Algorithm) plutôt qu'un tag, car un tag peut être déplacé. Le commentaire `# v7.0.1` donne la version, et Dependabot (dependabot.yml) propose les mises à jour.
- Cache npm, ligne 34 : `cache: npm`, clé calculée à partir de `package-lock.json`.
- `if: always()`, lignes 43 et 51 : le rapport est publié et les artefacts conservés même si les tests échouent.
- Le script `./run-tests.sh` est le même que dans le projet Java : la logique de test est dans un script et non dans le YAML, donc elle se rejoue en local et reste portable d'une CI à l'autre.

Le lien avec les manifestes Kubernetes (deployment-app.yaml:17) : l'image doit être tirée d'un registre (`TODO`), avec des `imagePullSecrets` pour un registre privé. Les `TODO` parlent du registre GitLab, car ils viennent du modèle d'exercice ; **le projet ayant choisi GitHub**, ce sera **GHCR** (`ghcr.io`). Il faudra ajouter à `ci.yml` un job qui construit et pousse l'image (avec `permissions: packages: write`). Pour tirer l'image depuis le cluster, le `GITHUB_TOKEN` ne suffit pas (il expire avec le job) : il faut un Personal Access Token avec `read:packages`.

Transposition de ce workflow vers GitLab CI (illustration)

`ci.yml` (GitHub)	Équivalent GitLab
`on: push, pull_request`	`workflow:rules` (pipeline sur branche et sur merge request)
`runs-on: ubuntu-24.04`	un runner, et `image: node:24-alpine`
`actions/checkout`	inutile : le dépôt est récupéré automatiquement

`ci.yml` **(GitHub)**	**Équivalent GitLab**
`cache: npm`	`cache:` avec `key:` basée sur `package-lock.json`
`run: ./run-tests.sh`	`script: [./run-tests.sh]`
`if: always()` + `upload-artifact`	`artifacts: when: always`
rapport JUnit (action tierce)	`artifacts:reports:junit`, intégré nativement
`concurrency`	`interruptible: true`
`permissions`	pas d'équivalent direct : droits du `CI_JOB_TOKEN` réglés dans le projet

À retenir

1. Même concepts (déclencheur, job, étapes ou script, artefacts, cache), mais un **modèle d'exécution différent** : GitHub partage la machine entre les étapes d'un job, GitLab isole chaque job.
2. L'ordonnancement diffère : **jobs parallèles +** `needs:` sur GitHub, **stages séquentiels** sur GitLab.
3. GitHub s'appuie sur un **écosystème d'actions** (à fixer par SHA pour la sécurité) ; GitLab sur `include:` / `extends:` et des fonctions intégrées (rapports JUnit, registre).
4. Mettre la logique dans un **script versionné** (`run-tests.sh`) rend le pipeline portable d'une plateforme à l'autre : seul le YAML d'enveloppe change.
5. Bonnes pratiques visibles dans ci.yml : droits minimaux, versions fixées, annulation des doublons, rapports publiés même en cas d'échec.